WebAssembly Microservices with Spin/wasmtime vs OCI Containers

Cold starts, isolation, throughput, and practical deployment guidance

---

LSC Project Team
Large Scale Computing 2026

44,471 movies **1** shared Rust core **2** runtime models

What we had to answer

Assigned topic

Deploy a microservice application using Spin and wasmtime, then compare cold-start latency, memory isolation, and throughput against equivalent OCI containers.

The real question

- Is Wasm practical as a serverless isolation substrate?
- What changes when the delivery unit is a WASI component instead of a container image?
- Which workloads actually benefit?

How we made it fair

- Same application behavior before measuring speed.
- Same local machine and benchmark harness.
- Results interpreted by workload shape, not by technology hype.

Quick vocabulary

WebAssembly

Portable binary instruction format.

Compiled Rust modules running outside the browser.

WASI

System interfaces for Wasm.

Controlled access to clocks, files, HTTP, randomness, and other host capabilities.

wasmtime

Runtime for Wasm components.

Executes WASI components and enforces the sandbox boundary.

Spin

Framework and CLI for Wasm microservices.

Provides HTTP triggers, local development, and deployment packaging.

OCI / Docker

Container image and runtime model.

Packages native binaries plus OS-level dependencies into a standard deployable image.

Two ways to deliver the same code

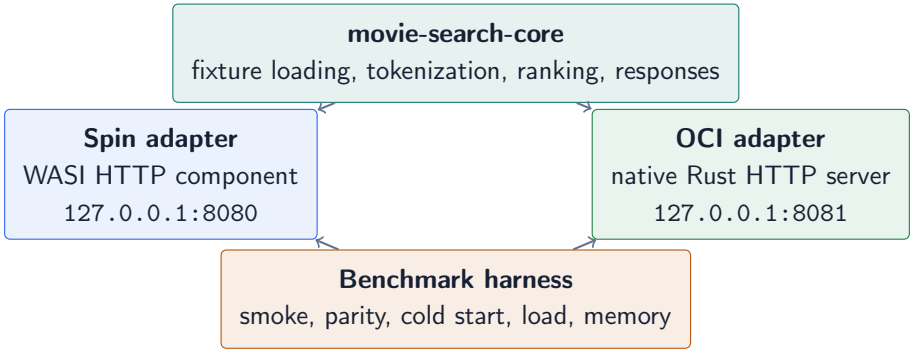
Spin / WASI component

- Application compiled to `wasm32-wasip2`.
- Host capabilities are explicit and narrow.
- Runtime boundary is the Wasm sandbox plus WASI.
- Good fit for request handlers, plugins, edge functions, and short-lived services.

OCI container

- Application compiled as a native binary.
- Dependencies are packed into an image filesystem.
- Runtime boundary is process, namespace, cgroup, and container runtime isolation.
- Good fit for compatibility, native libraries, storage engines, and mature operations.

What we built



44,471
movie documents after deduplication
same fixture

1 core
shared Rust application logic
same semantics

2 runtimes
Spin/wasmtime and OCI
same API

Why not benchmark full Meilisearch?

The initial idea was useful, but not fair enough

A direct "Meilisearch in Docker vs Meilisearch in Spin" comparison collapsed because the two sides could not share the same storage and ranking semantics.

Blockers we found

- Official Meilisearch uses LMDB and memory-mapped storage.
- Native dependencies such as crypto/runtime layers did not map cleanly to `wasm32-wasip2`.
- A custom Spin fallback would not be the same search engine.

Final decision

- Keep Meilisearch as feasibility evidence.
- Build a deterministic service with one shared core.
- Compare the isolation models, not two different applications.

Shared API and parity checks

Common endpoints

- `GET /health`
- `GET /version`
- `GET /stats`
- `GET /movies?offset=&limit=`
- `POST /search`

`benchmarks/compare_results.sh`

Parity queries

- `space`
- `toy story`
- `dark knight`
- `romance`
- empty query

Acceptance rule

Both runtimes must return the same engine identity, document count, total hits, and ordered `hit.id` values.

Benchmark methodology

1. **Correctness first**
   - unit tests
   - Spin build
   - Docker build
   - smoke and parity
2. **Performance**
   - 20 cold-start repeats
   - load at concurrency 10, 50, 100, 200
   - p50, p95, p99 latency
   - response validation
3. **Memory**
   - idle samples
   - under-load samples
   - Docker stats for OCI
   - host process RSS for Spin

Important caveat

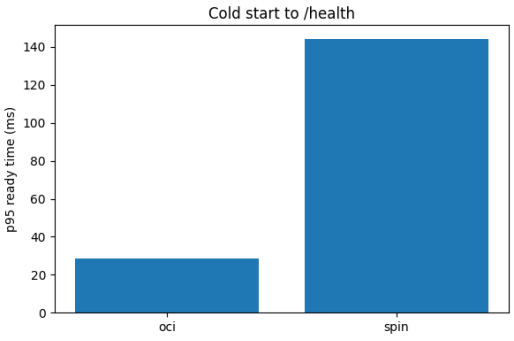
Memory accounting is operationally useful but not perfectly symmetric: Docker reports a container view, while Spin is measured as a host process tree.

Movie-search: cold start

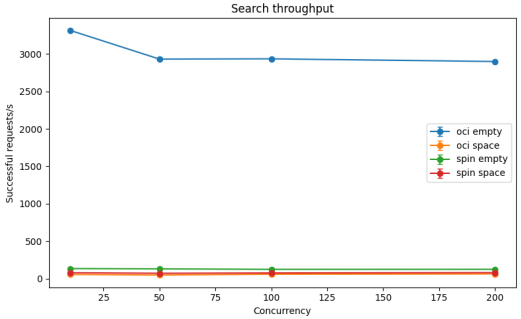
Runtime	Path	Median	p95
OCI	`/health`	21.0 ms	28.4 ms
Spin	`/health`	141.4 ms	144.2 ms
OCI	first search	307.8 ms	377.0 ms
Spin	first search	205.6 ms	217.5 ms

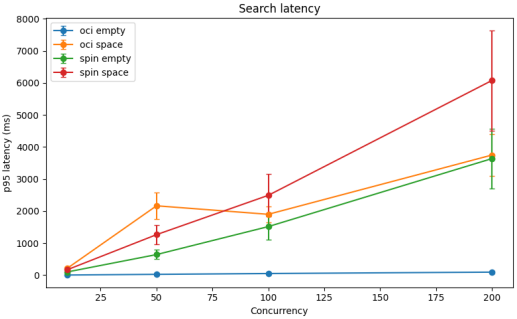
Interpretation

OCI reached a cheap readiness endpoint faster. Spin completed the cold path through a real search request faster in this implementation.



Movie-search: throughput and tail latency



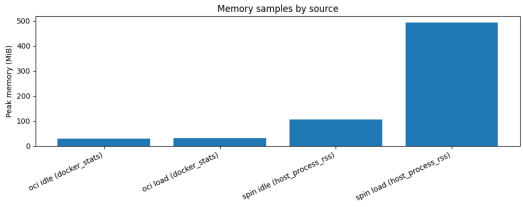


Empty query OCI was much faster on the low-application-cost path: 3315 req/s vs 133 req/s at concurrency 10.

space **query** Spin had higher c=10 throughput, 77 req/s vs 54 req/s, but worse tails at high concurrency.

Movie-search: memory is not automatically lower

Runtime	Phase	Max observed
OCI	idle	29.8 MiB
OCI	load	31.9 MiB
Spin	idle	104.8 MiB
Spin	load	493.5 MiB



Takeaway

Wasm gives a different isolation model. It does not guarantee lower RSS for every host, runtime, and workload.

Second workload: file-tools

Spin implementation

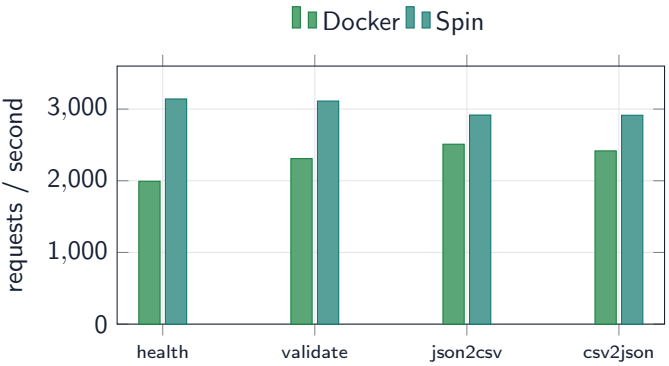
- `spin-file-tools-sdk4`
- Rust component with `spin-sdk = 4`
- Default port: 3000
- Same request-level features as Docker side

Docker implementation

- `docker-file-tools`
- Native Rust server with Axum
- Default port: 8081
- Built as a Docker image

`/validate/json` `/convert/json-to-csv` `/convert/csv-to-json` `/image/metadata` `/image/grayscale` `/image/resize`

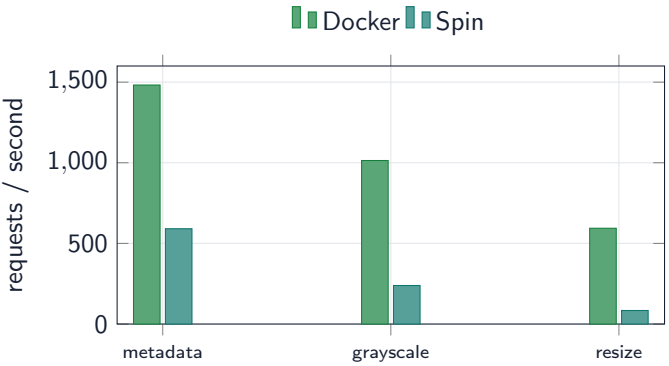
File-tools: light request paths at concurrency 50



Result pattern

- Spin was faster on these light paths.
- Throughput ratio ranged from 1.16x to 1.58x over Docker.
- p95 latency was similar or slightly better for Spin.

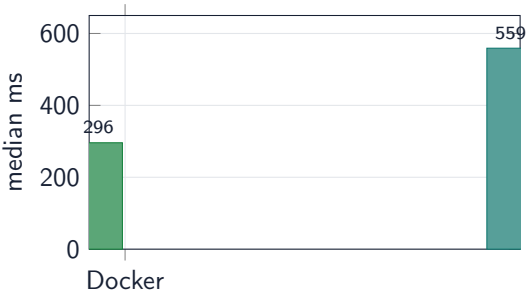
File-tools: image-heavy paths at concurrency 50



Result pattern

- Docker was much faster for image processing.
- Resize at c=50: 594 req/s vs 84 req/s.
- Spin p95 resize latency reached about 799 ms.

File-tools: cold start snapshot



Runtime	n	Median	Range
Docker	5	296 ms	264-497
Spin	5	559 ms	553-561

How to read this

These file-tools cold-start samples are smaller than the main benchmark, so they support the practical story but do not replace the movie-search benchmark.

What the two workloads taught us

Spin/wasmtime looked attractive when

- the handler was small and request-scoped;
- the workload stayed close to parsing, routing, and lightweight transformation;
- sandboxing and portability mattered more than native library reach;
- the deployment unit could be a capability-limited component.

OCI looked stronger when

- native code paths and mature libraries dominated runtime cost;
- storage, memory mapping, or process tooling mattered;
- resource accounting had to match operations tooling;
- compatibility and ecosystem maturity were more important than minimal capability exposure.

Recommendations

Use Spin/Wasm for

- edge/serverless handlers;
- plugin systems;
- untrusted extensions;
- small API adapters;
- capability-limited jobs.

Use OCI for

- storage-heavy services;
- native databases;
- CPU-heavy media work;
- complex dependencies;
- standard production operations.

Always benchmark

- cold path and warm path;
- low-cost and real endpoints;
- tail latency;
- memory from the same viewpoint where possible;
- correctness under load.

Demo path

Movie-search

```
cd spin-meili
spin build
spin up --listen 127.0.0.1:8080

cd oci-movie-search
docker compose up --build

benchmarks/compare_results.sh
```

File-tools

```
curl http://localhost:3000/routes
curl http://localhost:8081/routes

curl -X POST /validate/json
curl -X POST /image/resize
```

Fallback

Show saved CSV files in `results/processed` and `bench-results`, then open the plots in `results/plots`.

Final conclusions

- We ran equivalent microservices in Spin/wasmtime and OCI with validated functional parity.
- Wasm is practical for some serverless-style microservices, especially small isolated request handlers.
- OCI remains the safer default for native, storage-heavy, and operations-heavy services.
- "Wasm is lighter than containers" is too simple: workload shape changed the result.

Bottom line

Choose the isolation substrate after measuring actual endpoint behavior.

Same code first.
Measurements second.
Recommendations last.